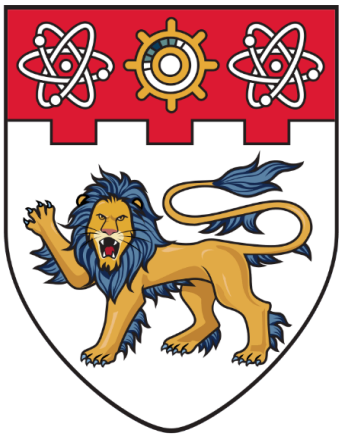


NANYANG TECHNOLOGICAL UNIVERSITY

SINGAPORE

SC3040 Advanced Software Engineering:

Lab 01 Project Proposal

Version <1.1>

Project Manager (Overall delivery of product)	Gary Quah YiKai
Lead Developer (Overall technical lead, responsible for technical aspects of product release)	Matthew Wong
QA Manager (Overall product and process quality, implementation of QA processes)	Lau Jing Jie
QA Engineers (Devise test plans, conduct tests)	Lee Zheng Wee Rusyaidi
Release Engineer/Manager (Create baselines and build and integrate changes for delivery)	Quah Yew Yi
Full stack developers (Participate in the entire SDLC, generating work products including documentation, source code, unit and integration tests)	All team members

Revision History

Revision Number	Date	Primary Author(s)	Comments
0.1	Aug 27th, 2026	Gary Quah, Lau Jing Jie, Lee Zheng Wee, Rusyaidi, Quah Yew Yi, Matthew Wong	Initial draft
0.2	Sept 2nd, 2026	Gary Quah, Lau Jing Jie, Lee Zheng Wee, Rusyaidi, Quah Yew Yi, Matthew Wong	Updating of draft
1.0	Sept 09th, 2026	Gary Quah, Lau Jing Jie, Lee Zheng Wee, Rusyaidi, Quah Yew Yi, Matthew Wong	Completion of edits for initial report.
1.1	Nov 4th, 2026	Gary	Update project functional requirements for homepage after rescope.

Table of contents

Revision History **2**
1. Introduction **4**
1.1. Purpose 4
1.2. Intended Audience 4
1.3. Objectives 4
1.4. Scope 4
2. Project functions **5**
2.1. Functional requirements 5
2.1.1. Login/Registration page 5
2.1.2. Friend page 5
2.1.3. Home page 5
2.1.4. Settings page 6
2.1.5. Meal analysis page 6
2.1.6. Chatbot page 8
2.1.7. Training routine page 9
2.1.8. Gamification page 10
2.2. Non-functional requirements 10
2.2.1. Performance 10
2.2.2. Scalability 10
2.2.3. Maintainability 10

2.2.4. User experience 10
2.2.5. Security 11
3. Use case diagrams 12
3.1. System overview 14
3.2. Login/Registration page 15
3.3. Friend page 15
3.4. Home page 16
3.5. Settings page 17
3.6. Meal analysis page 18
3.7. Chatbot page 19
3.8. Training routine page 20
3.9. Gamification page 21
4. Project stack 23
4.1. Frontend 23
4.1.1. Next.js 23
4.1.2. Tailwind CSS 24
4.1.3. Rechart & Lucide React 24
4.2. Backend: 24
4.2.1. Postgres (Supabase) 24
4.2.2. NodeJS 25
4.2.3. FastAPI 25
4.3. API 25
4.3.1. Gemini 25
4.3.2. Strava API 25

1. Introduction

1.1. Purpose

The primary purpose of the project is to develop a smart fitness tracker web application that provides personalised training scheduling and meal planning.

The application aims to assist users in planning their workouts and nutrition based on their personal health, fitness and daily food consumption data by leveraging Artificial Intelligence (AI) and data analytics to provide personalised recommendations.

1.2. Intended Audience

The target audience for this application includes people of different ages and fitness levels who are keen on improving their health and lifestyles through technology-assisted fitness and nutrition management.

1.3. Objectives

The main objectives of this project are:

a. To design and develop a fitness tracker incorporating Artificial Intelligence (AI) and data analytics
b. To demonstrate sound software engineering skills

1.4. Scope

The scope of this project includes:

a. Development of a full-stack web application consisting of both frontend and backend components.
b. Frontend implementation for user interaction, input handling, and result visualisation.
c. Backend implementation for business logic, data management, and integration with Gen-AI services or other external API services.

2. Project functions

2.1. Functional requirements

2.1.1. Login/Registration page

2.1.1.1. **User Registration:** The system shall allow users to create an account.

2.1.1.2. **User Login:** The system shall authenticate users using their credentials.

2.1.1.3. **User Logout:** The system shall allow authenticated users to log out of their account

2.1.2. Friend page

2.1.2.1. **View Friends:** The system shall allow users to view their current friends.

2.1.2.2. **Add Friend:** The system shall allow users to search for and add other users as friends.

2.1.2.3. **Remove Friend:** The system shall allow users to remove existing friends.

2.1.3. Home page

2.1.3.1. **User data modification:** The user interface on the homepage shall allow users to manually modify their health data, such as height and weight.

2.1.3.2. **Health data overview:** The user interface on the homepage shall display visual summaries corresponding to the user's health data, such as height, weight and body mass index.

2.1.3.3. **Training overview:** The user interface on the homepage shall display a visual summary of the user’s training activity, such as mileage and heart rate. The user shall have the option to select which data to display: daily or latest training data.

2.1.3.4. Food consumption overview: The user interface on the homepage shall display a visual summary of the user’s calorie

intake. The user shall have the option to select which data to display: weekly or daily calorie intake.

2.1.3.5. **Daily check-in:** The user interface on the homepage shall provide a button for users to perform a daily check-in, along with a user interface to display the number of consecutive days checked in.

2.1.4. Settings page

2.1.4.1. Profile & TDEE Engine

2.1.4.1.1. **Biological metrics:** The application must allow users to input their biological metrics: age (int, years), weight (float, kg), height (float, cm), biological sex (string, male/female)

2.1.4.1.2. **Activity levels:** The application must allow users to select their current activity level (Sedentary, Lightly Active, Moderately Active, Very active, Extra Active) and their fitness goal (Deficit, Maintenance, Surplus).

2.1.4.1.3. Health statistics:

2.1.4.1.3.1. Upon submission of user's profile metrics, the application must calculate their Basal Metabolic Rate (BMR) and Total Daily Energy Expenditure (TDEE) using Mifflin-St Jeor equation

2.1.4.1.3.2. The application must calculate the daily macronutrient (grams of protein, Carbs, Fats) based on computed TDEE and their activity level

2.1.4.1.4. Strava OAuth Integration:

2.1.4.1.4.1. The application must provide a "Link Strava" UI that redirects unauthenticated clients to the official Strava OAuth 2.0 authorization portal.

2.1.4.1.4.2. The backend shall receive a temporary access_token and persistent refresh_token via HTTP POST request to Strava API

2.1.5. Meal analysis page

2.1.5.1. Media Capture & Ingestion:

2.1.5.1.1. The system shall allow users to upload meal images via local file selection.

2.1.5.1.2. The system shall support common web image formats: JPEG, PNG, WEBP and HEIC/HEIF (with client-side conversion for HEIC where needed).

2.1.5.1.3. The user shall have the option to attach a short supplementary text description or voice note transcript (e.g., "cooked with 1 tbsp butter", "whole milk used") before initiating the analysis.

2.1.5.2. AI Vision & Multimodal Extraction:

2.1.5.2.1. The backend shall extract an itemized breakdown of food components from the uploaded image.

2.1.5.2.2. For each identified item, the system shall estimate:

a. Component name (e.g., Grilled Chicken Breast)
b. Estimated portion weight in grams (g)
c. Total calories (kcal)
d. Macronutrient profile: Protein (g), Carbohydrates (g), and Fat (g)

2.1.5.2.3. The system shall aggregate itemized values into total meal calories and macronutrient totals.

2.1.5.2.4. The system shall provide an inference confidence score (0.0 to 1.0) representing visual certainty.

2.1.5.2.5. The model shall generate contextual athletic feedback evaluating the meal against endurance energy demands (e.g., glycogen replenishment, protein synthesis timing relative to workout load).

2.1.5.3. Human-in-the-Loop Review & Modifications

2.1.5.3.1. The user interface shall display the estimated macronutrients and itemized list in an editable preview state prior to database persistence.

2.1.5.3.2. The user shall be able to:

2.1.5.3.2.1. Adjust serving sizes/weights (which automatically recalculates the item's and total meal's macros proportionally).

2.1.5.3.2.2. Add missed food items manually.

2.1.5.3.2.3. Delete incorrectly identified ingredients.

2.1.5.3.2.4. Reclassify the meal category (Breakfast, Lunch, Dinner, Snack).

2.1.5.4. Daily Food Logging & Aggregation

2.1.5.4.1. Upon user confirmation, the system shall write the finalized meal log to the database (meals table) linked to the authenticated user's ID.

2.1.5.4.2. The system shall update the athlete’s rolling daily total for calories, protein, carbohydrates, and fats.

2.1.5.4.3. The system shall compute the net energy balance against daily energy expenditure: Net Energy = Calories In - BMR + Strava Active Calorie Burn

2.1.6. Chatbot page

2.1.6.1. Conversational Interface

2.1.6.1.1. The application must provide a text-based input and submit button for users to query the chatbot using natural language regarding their fitness journey

2.1.6.1.2. The chatbot interface must display a scrollable, chronological history of active chats.

2.1.6.1.3. The chatbot interface must visually differentiate between user prompts and generated responses

2.1.6.2. Context

2.1.6.2.1. The backend service must append the authenticated user’s data, such as TDEE, fitness goals, logged meals, and exercise history, as hidden context to the LLM prompt

2.1.6.2.2. The chatbot must analyse user feedback and propose actionable insights into their training routine

2.1.7. Training routine page

2.1.7.1. Training plan generation

2.1.7.1.1. **Training preferences:** The application must allow users to specify their training days per week (int, 1-7), preferred session duration (int, minutes), and preferred activity types (run, gym, cycling, bodyweight).

2.1.7.1.2. **Plan generation**: The system shall generate a personalised multi-week training plan based on the user's profile metrics, fitness goal, training preferences, and imported training history.

2.1.7.1.3. **Progressive overload constraint**: The generated plan must increase weekly training volume by no more than a configurable safe threshold between consecutive weeks.

2.1.7.2. Readiness scoring

2.1.7.2.1. **Intensity classification:** The system shall classify each day into one of three recommended intensity levels: deload, light, or push.

2.1.7.3. Daily recommended workout

2.1.7.3.1. **Workout display**: The system shall display the recommended workout for the current day, including exercises or target distance, target intensity, and estimated session duration.

2.1.7.3.2. **User response**: The system shall allow the user to accept, swap, or skip the recommended workout for that day.

2.1.7.3.3. **Session logging**: The system shall record completed workouts back into the user's training history for use in subsequent readiness calculations.

2.1.8. Gamification page

2.1.8.1. **Experience Point (XP) System:** The system shall award users Experience Points (XP) upon successful completion of training routines, daily check-ins, and consistent meal logging.

2.1.8.2. **Dynamic Leaderboards:** The system shall display an interactive leaderboard ranking the user against their friends based on accumulated XP, running mileage, and streak consistency.

2.1.8.3. **Shared User Challenges:** The system shall allow users to create, invite friends to, and participate in shared fitness challenges.

2.1.8.4. **Interactive Daily Check-in:** The system shall prompt users with a short, gamified daily questionnaire to log subjective metrics such as muscle fatigue, mood, and diet adherence. Completion of this check-in shall increment the user's login streak and award daily bonus XP.

2.2. Non-functional requirements

2.2.1. Performance

2.2.1.1. The application shall provide fast response times by optimising code where possible.

2.2.2. Scalability

2.2.2.1. The system shall be designed to handle an increasing number of users, requests and data without performance degradation.

2.2.3. Maintainability

2.2.3.1. The application shall follow architecture patterns where, for the frontend, data logic is separated from the user interface, while backend services and database operations are organised into separate layers, enhancing maintainability and testability.

2.2.4. User experience

2.2.4.1. All actions performed by the user shall have a form of feedback to let the user be aware that processes are happening in the background.

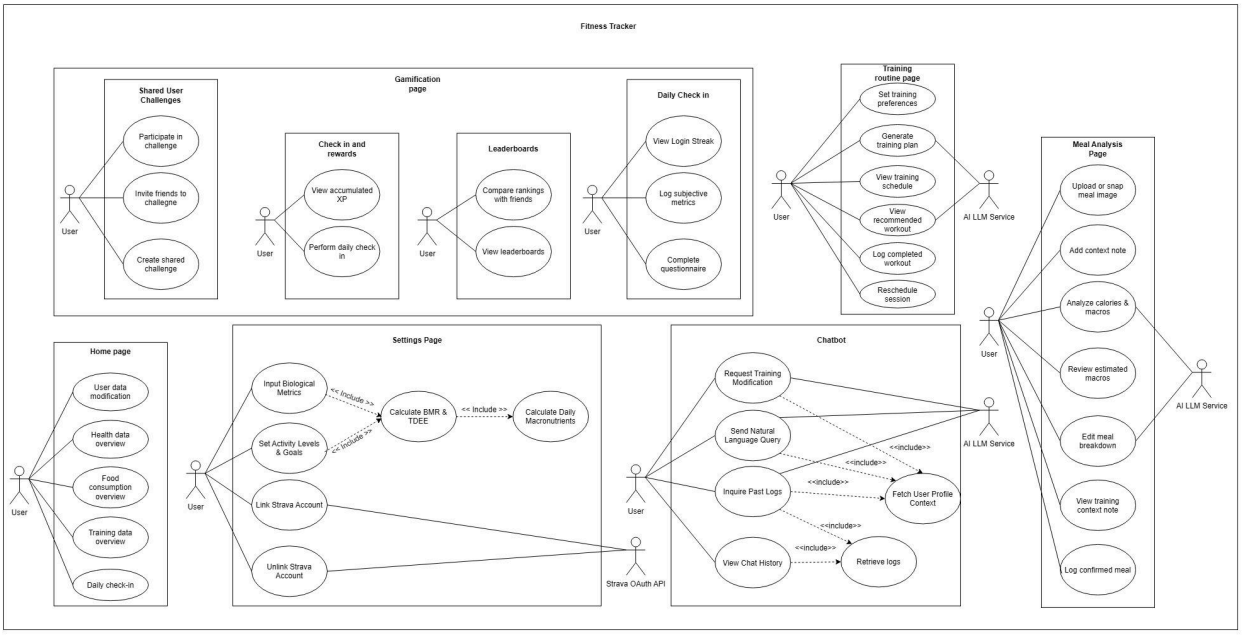
2.2.5. Security

2.2.5.1. The system shall securely authenticate users and protect user account information. It shall also restrict access to authenticated features to logged-in users.

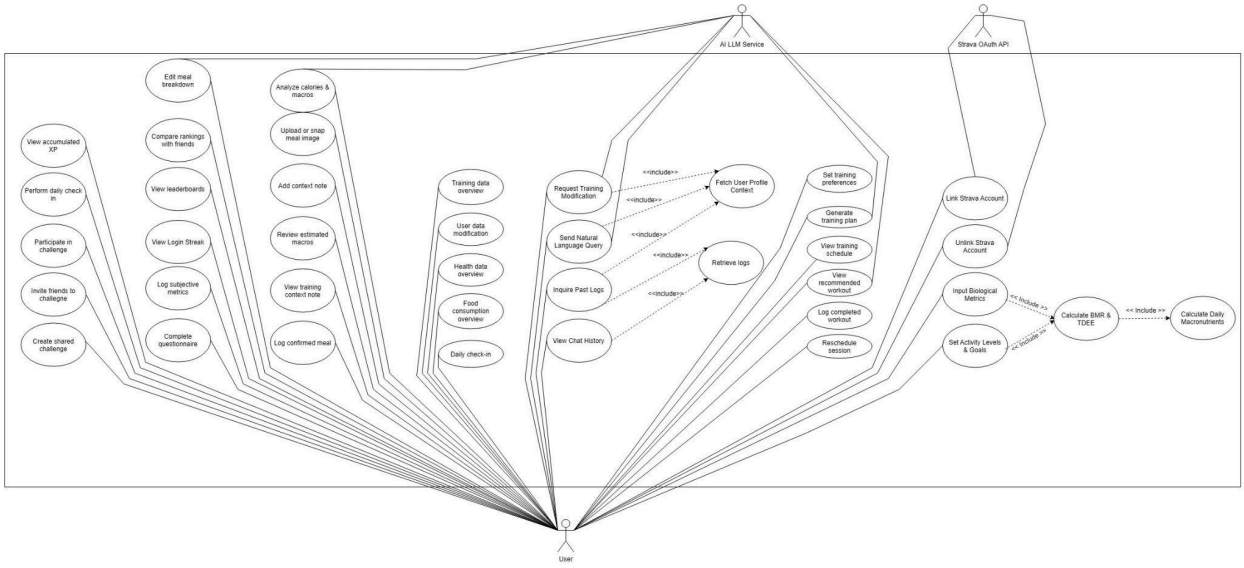
3. Use case diagrams

The use case diagrams are abstracted to allow immediate clarity behind the individual systems and pages.
https://drive.google.com/file/d/1trnxbpELbxK88P3Sp8ze1p7i74GjV2y3/view?usp=sharing

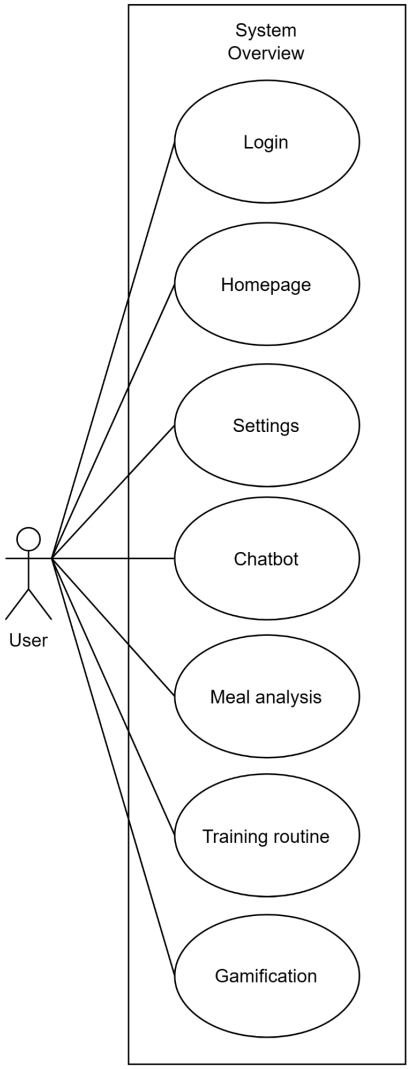
Abstracted use case diagrams:



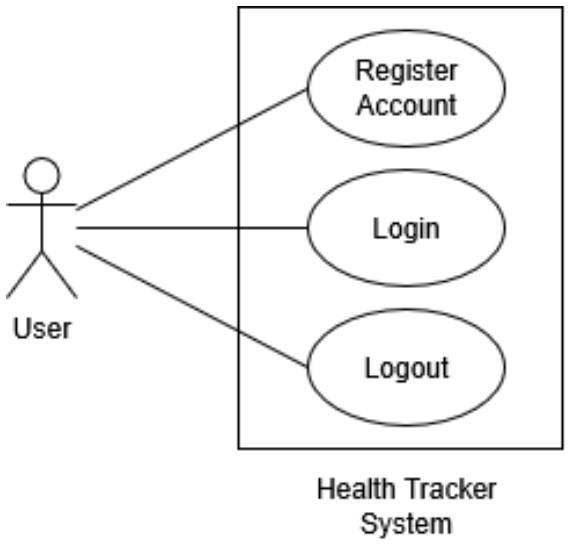
Combined use case diagrams:


AI LLM Service
Strava OAuth API
View accumulated XP
Perform daily check in
Participate in challenge
Invite friends to challegne
Create shared challenge
Edit meal breakdown
Compare rankings with friends
View leaderboards
View Login Streak
Log subjective metrics
Complete questionsaire
Analyze calories & macros
Upload or snap meal image
Add context note
Review estimated macros
View training context note
Log confirmed meal
Training data overview
User data modification
Health data overview
Food consumption overview
Daily check-in
Request Training Modification
Send Natural Language Query
Inquire Past Logs
View Chat History
<<include>>
Fetch User Profile Context
Retrieve logs
Set training preferances
Generate training plan
View training schedule
View recommended workout
Log completed workout
Reschedule session
Link Strava Account
Unlink Strava Account
Input Biological Metrics
Set Activity Levels & Goals
<< Include >>
Calculate BMR & TDEE
Calculate Daily Macronutrients
User

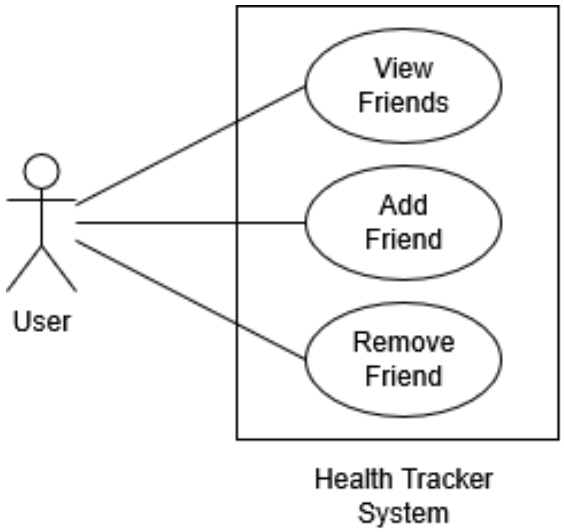
3.1. System overview



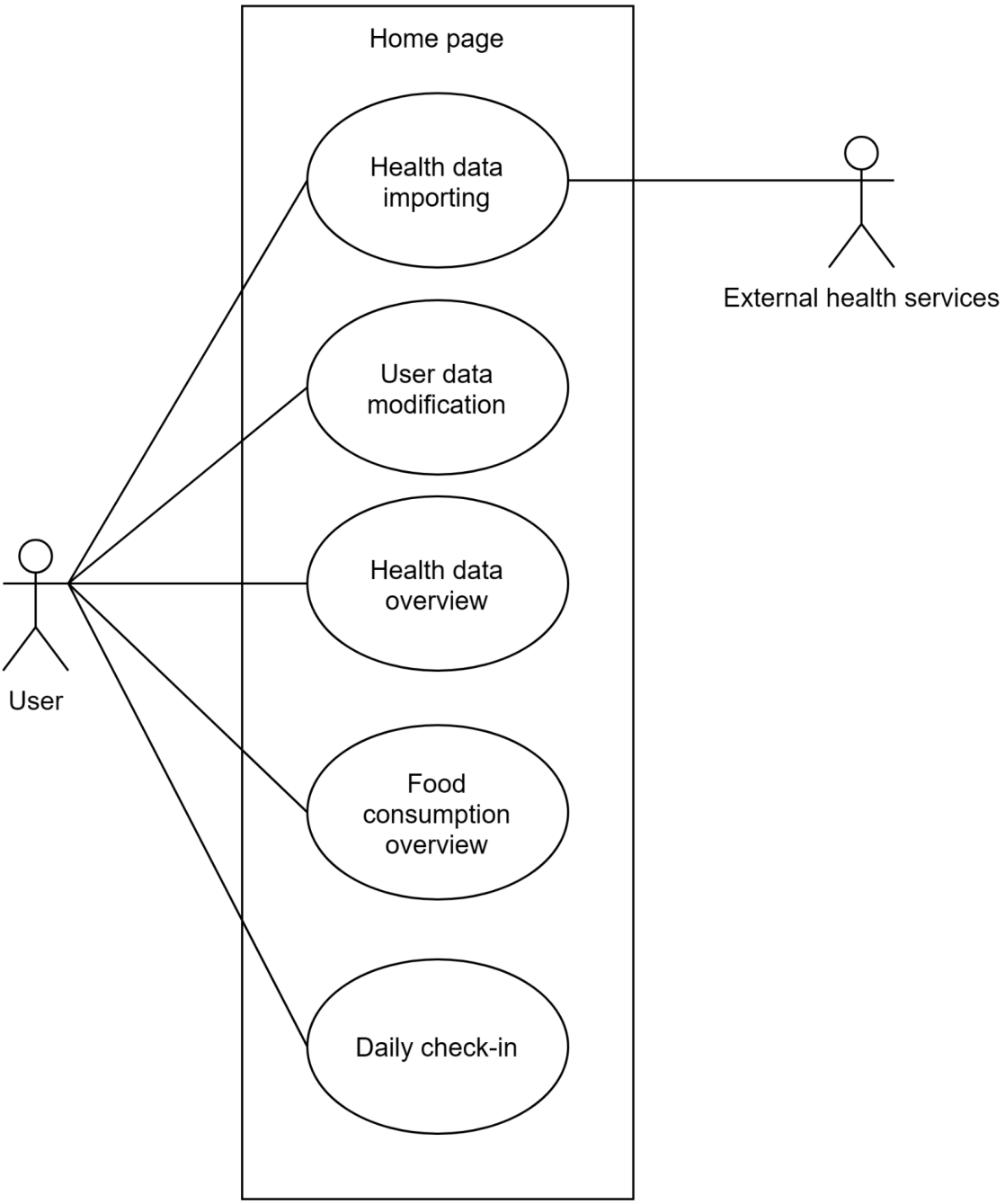
3.2. Login/Registration page



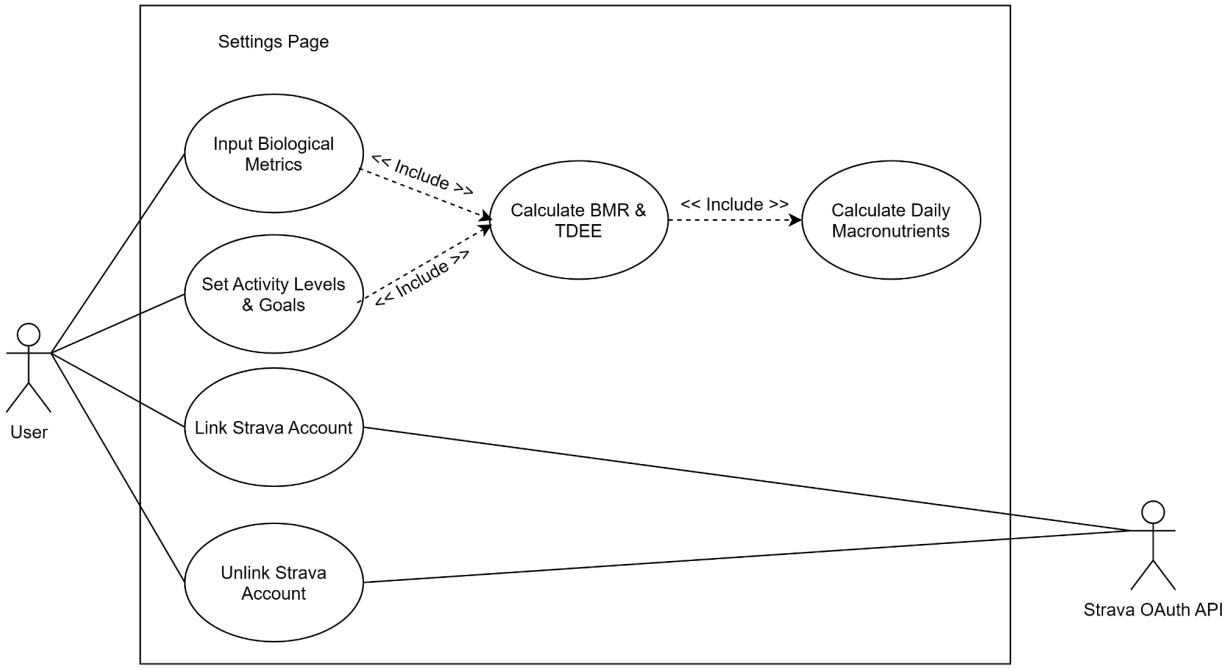
3.3. Friend page



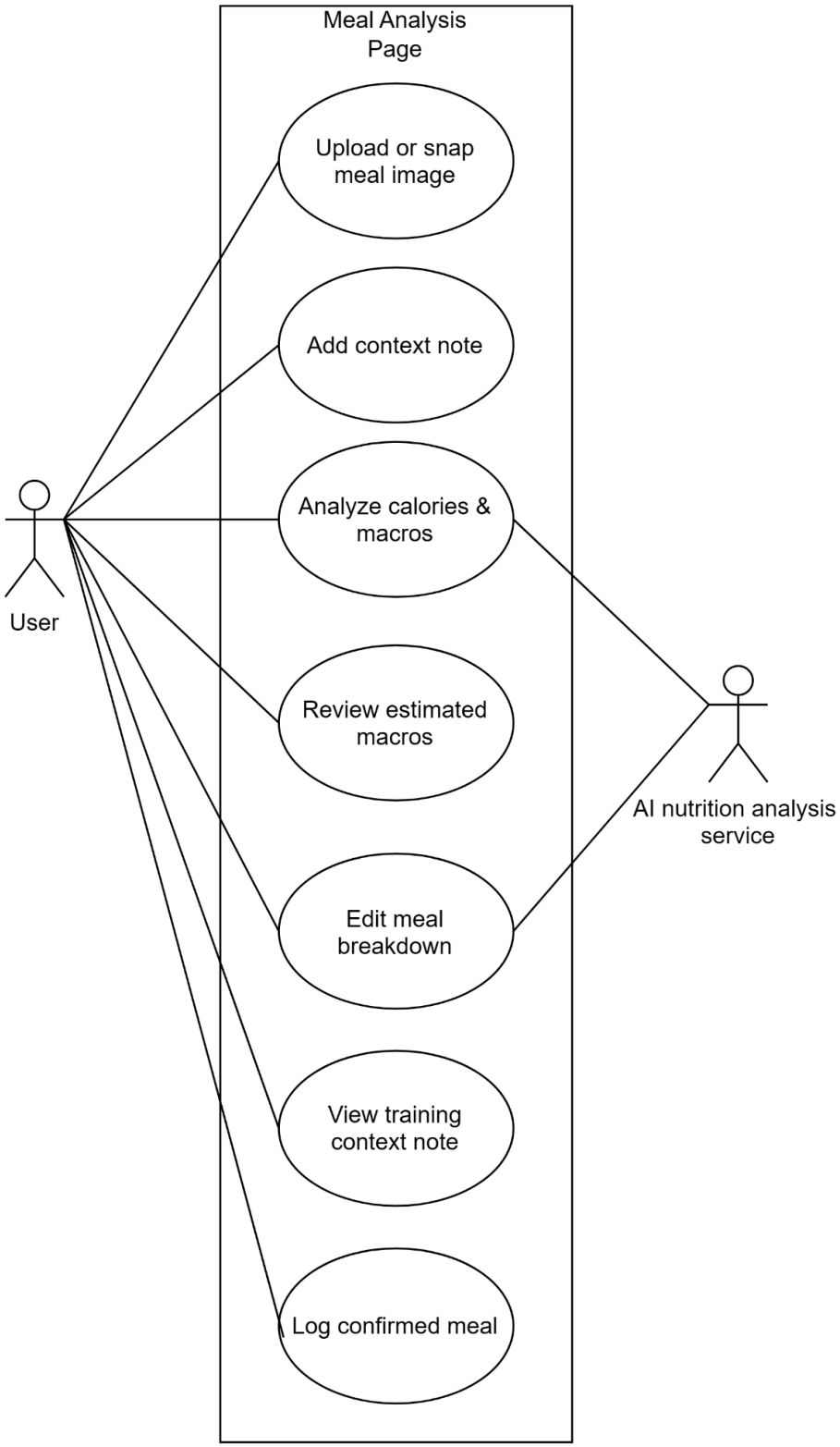
3.4. Home page


Home page
Health data importing
User data modification
Health data overview
Food consumption overview
Daily check-in
User
External health services

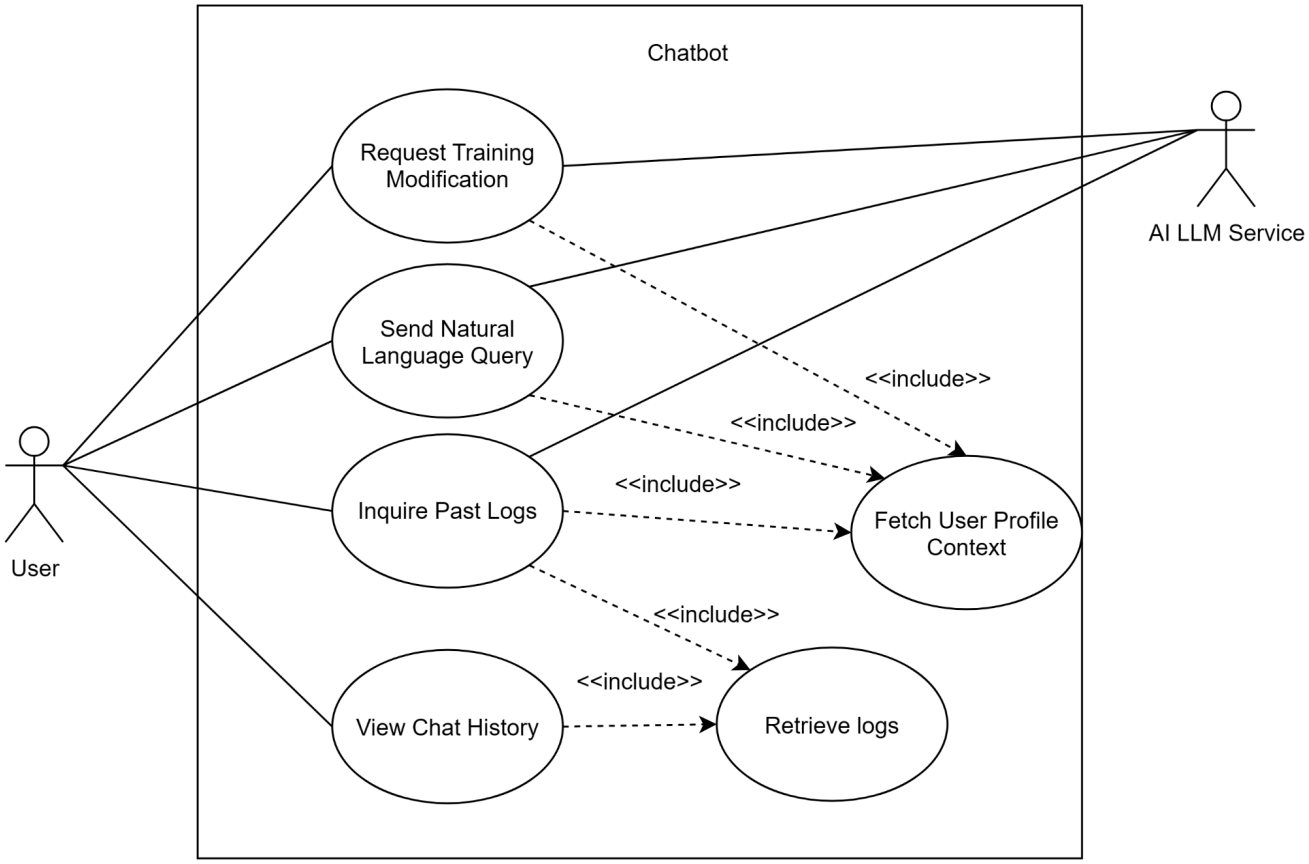
3.5. Settings page



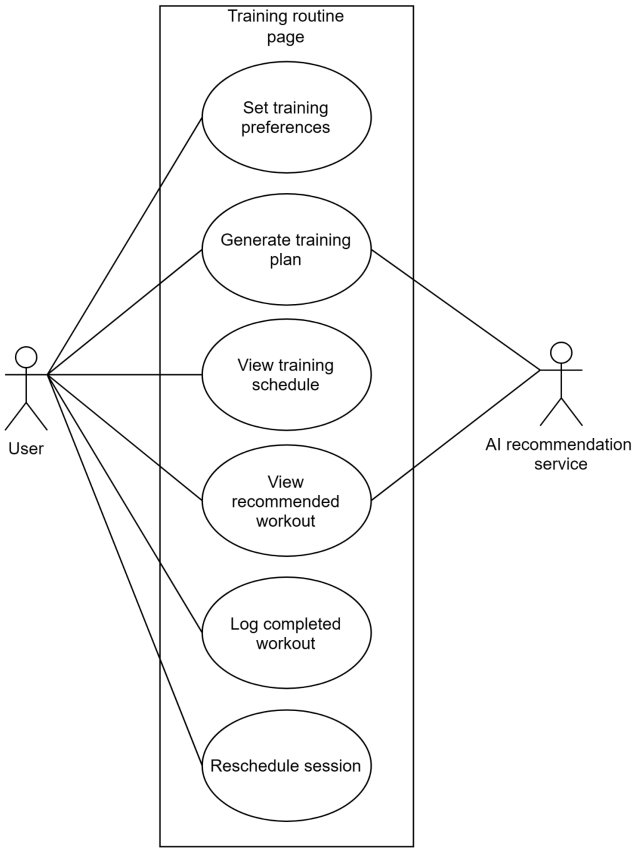
3.6. Meal analysis page



3.7. Chatbot page

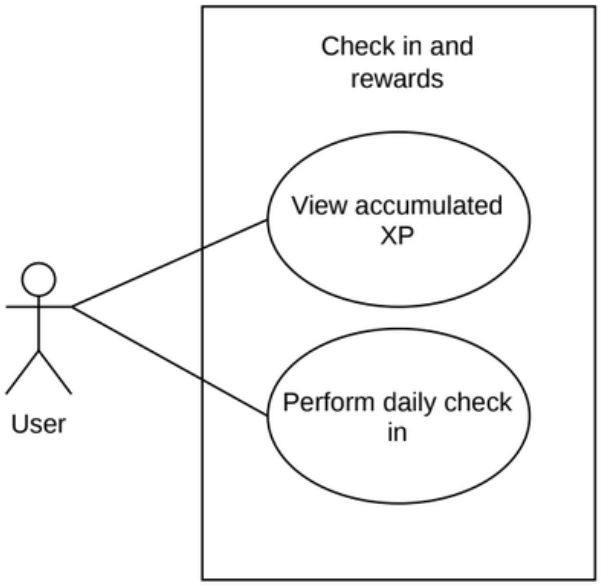


3.8. Training routine page

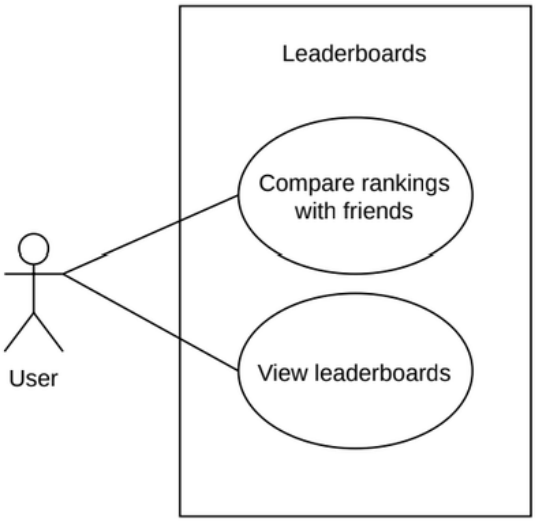


3.9. Gamification page

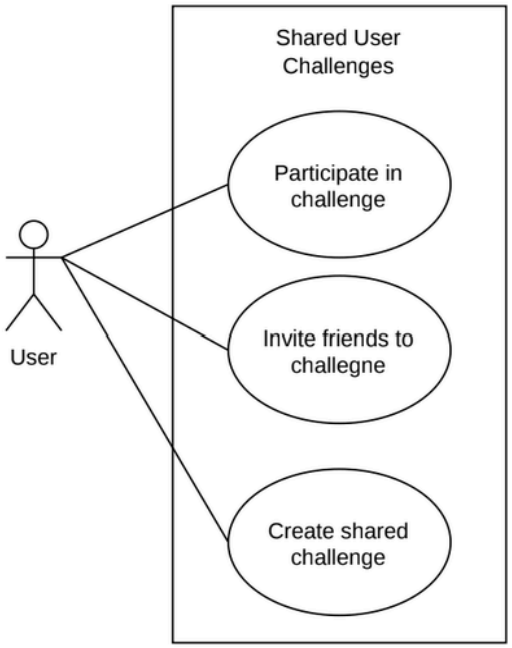
3.9.1. Experience Point (XP) System



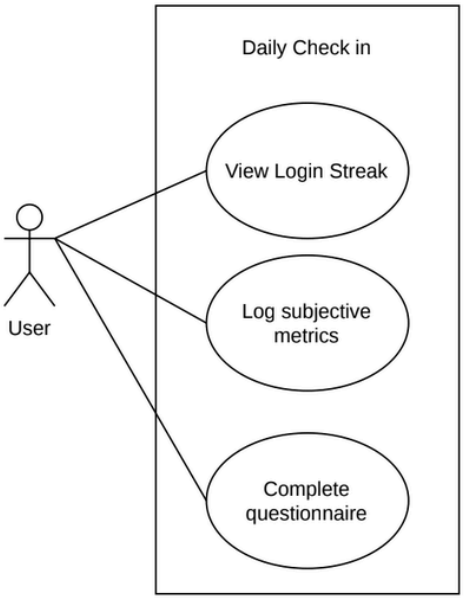
3.9.2. Dynamic Leaderboards



3.9.3. Shared User Challenges



3.9.4. Interactive Daily Check-in



4. Project stack

The proposed full-stack project architecture is designed to deliver responsive, scalable and intelligent fitness tracking experiences for users. The overall application consists of a web-based frontend, backend services, persistent data storage and an external application programming interface (API) for users' data.

4.1. Frontend

The frontend serves as a presentation layer for users to interact with the application. The chosen frontend stack must be responsive and deliver an intuitive user experience.

4.1.1. Next.js

NextJS is a full-stack web development framework which is built on top of the React JavaScript library. While React is mainly used to develop user interfaces on the frontend, NextJS handles both the frontend rendering and backend logic simultaneously.

NextJS supports pre-rendering techniques such as server-side rendering (SSR) and static-site generation (SSG), which can improve initial page load performance, allowing frequently accessed pages and content to be delivered efficiently.

NextJS provides file-based routing, allowing pages to be organized and accessed through a structured uniform resource locator (URL) system. Additionally, it also handles authenticated requests and communication with backend services securely. This is especially important for the application due to its many API and backend services.

4.1.2. Tailwind CSS

Tailwind CSS is a utility-first CSS framework used for styling the user interface. Rather than relying on rigid component libraries or stylesheet abstractions, Tailwind allows usage of utility classes to be composed directly within JSX markup.

This facilitates a consistent, responsive design system optimized for dark-mode dashboards. Tailwind v4's high-performance compilation engine produces a minimal production CSS footprint, ensuring optimal rendering speed on mobile and desktop viewports. This is particularly useful for our application as users may access the platform through desktops, tablets and mobile devices. Tailwind CSS promotes consistency across the application and allows the user interface to be developed efficiently.

4.1.3. Rechart & Lucide React

To convey complex sports science data, the frontend uses Recharts, a React charting library on SVG. Rechart renders dynamic acute vs chronic workload curves, rolling fatigue ratios, and heart rate zone distributions. Lucide React provide a standardised icon system for navigation, biometric status indications and workout categories.

4.2. Backend:

The backend refers to the server-side development of an application containing operations such as retrieval and sending data from a database to the user interface.

4.2.1. Postgres (Supabase)

Persistent data is managed via PostgreSQL, an object-relational database management system hosted on Supabase. PostgreSQL provides strict ACID

compliance and relational integrity across athlete profiles, workout streams, daily aggregated training loads, and historical AI recommendations.

4.2.2. NodeJS

Node.js provides an event-driven, non-blocking I/O runtime well-suited for high-throughput API endpoints and asynchronous webhooks. Server-side Route Handlers (app/api/) and Server Actions (app/actions/) serve as the application's RESTful endpoints and secure controller layer. This full-stack architecture eliminates serialization boundaries between frontend and backend, enabling end-to-end type safety with TypeScript and Zod validation schemas.

4.2.3. FastAPI

FastAPI is a modern web framework for building Application Programming Interfaces (API) with Python. Its syntax is fast to use and intuitive, making development fast and less error-prone.

4.3. API

4.3.1. Gemini

The platform integrates Google’s Gemini 2.5 Flash model. By feeding structured 7-day and 28-day ACWR trends, resting heart rate baselines, and workout summaries into strongly typed prompts, the model generates structured, scientifically grounded coaching recommendations, detailing prescribed workout types, heart rate target zones, recovery protocols, and physiological rationales.

4.3.2. Strava API

Strava API acts as the primary athletic telemetry data source. The application uses Strava’s OAuth 2.0 protocol for athlete authorization, automated token refresh lifecycles, and real-time Webhook Subscriptions. Upon workout completion, high-resolution time, cadence, and heart-rate telemetry streams are downloaded for parsing.

These data from Strava will be used in data analytics in conjunction with AI to create customised training plans.